

Multi-Agent Debate System for News Credibility: Architecture and Usage Guide

Gideon Hale

1 SYSTEM ARCHITECTURE OVERVIEW

The system takes in metadata about news articles and runs it through a structured multi-agent debate system, which ultimately returns a score on the reliability of the news article.

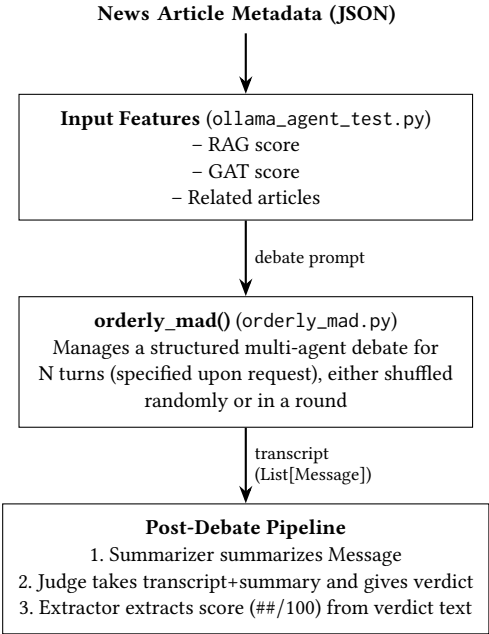


Figure 1: System Event Timeline and Flow

The workflow is summarized as follows:

- (1) **Input Feature Extraction:** An input metadata file of a news article (in JSON format) is parsed to extract retrieval-augmented generation (RAG) signals, graph attention network (GAT) scores, and related articles.
- (2) **Multi-Agent Debate:** The `orderly_mad()` engine runs an N -turn debate among randomized agents reacting to the prompt.
- (3) **Post-Debate Pipeline:** A *Summarizer* synthesizes the debate transcript; a *Judge* renders a verdict message based on the transcript and summary; and an *Extractor* parses the verdict to produce a normalized score from 0 to 100.

2 INPUT DATA SUMMARY

A summary of all the major input fields that the system takes in and feeds to the MAD system are outlined in [Table 1](#).

3 SYSTEM COMPONENTS

3.1 Core Agent Abstraction (`ollama_agent.py`)

This module defines the foundational data types and agent behavior:

Table 1: Summary of Input Processed Signals

Signal	Source	Range	Purpose
Source Score	Adfontes	0.0–1.0	Avg. source reliability
Missing Rate	RAG System	0.0–1.0	Coverage gap indicator
Num Articles	RAG System	0–30	Corroboration volume
Num Unrated	RAG System	0–30	Unknown-source volume
GAT Score	Comment Graph	0.0–1.0	Topological credibility

- **Message:** A data class representing a single conversational turn with role, content, and author.
- **Agent:** A data class encapsulating a named persona with a system prompt, model identifier, temperature, and an OpenAI-compatible client. The `respond` method builds context and fetches the conversational LLM response.

3.2 Instantiated Agent Pool (`agent_presets.py`)

Configures and exports all named agents used in experiments, sharing an Ollama-backed endpoint. The primary agents mapped to the *Journalist* persona and utility tools are outlined in [Table 2](#).

Table 2: Agent Roles and Configurations

Agent	Role	Model
Joseph	Journalist (investigative)	<code>mistral:8x7b</code>
Steven	Journalist (objective)	<code>gpt-oss:120b</code>
Benjamin	Journalist (community)	<code>qwen2.5:72b</code>
Christopher	Journalist (educational)	<code>mistral:8x7b</code>
Elijah	Journalist (misinfo specialist)	<code>gpt-oss:120b</code>
Summarizer	Neutral observer	<code>qwen2.5:72b</code>
Judge	Unbiased verdict-giver	<code>qwen2.5:72b</code>
Extractor	Score extractor (##/100)	<code>mistral:8x7b</code>

3.3 Role Definitions (`roles.py`)

Provides a dictionary of system prompt templates keyed by specific domains such as journalist, mathematician, public administrator, and parent. Each role contains multiple persona variants providing ideological and methodological diversity within the debate. For the core evaluation of news articles, the system utilizes the "Journalist" domain, which defines five distinct personas. Each persona is assigned specific pedagogical instructions (task descriptions) that alter their approach to credibility assessment, as detailed in [Table 3](#).

3.4 Debate Strategy and Orchestration (`orderly_mad.py`)

The `orderly_mad()` engine is responsible for the synchronous execution of the multi-agent debate. It takes a discussion prompt,

Table 3: Journalist Personas and Task Descriptions

Persona	System Prompt / Task Description
Investigative	Focuses on holding the powerful accountable and exposing injustice through investigative reporting.
Objective	Values objectivity and fairness, striving to present a balanced and impartial account of events.
Community	Prioritizes community engagement and storytelling, amplifying the voices of ordinary people.
Educational	Believes the media has a responsibility to educate the public and rigorously promote informed discourse.
Misinformation Specialist	Excels specifically at identifying fake news, logical fallacies, and deliberate misinformation.

a list of agents, a maximum round count, and an agent ordering strategy.

A critical vulnerability in multi-agent LLM systems is “anchoring bias”—a phenomenon where the first agent to speak disproportionately influences the consensus trajectory of the ensuing debate. To mitigate this, `orderly_mad()` implements dynamic strategies that allow for randomized permutation (‘shuffle’) rather than strictly sequential execution (‘round’). By shuffling the order of speakers per article, the system ensures that dissenting or highly critical personas (e.g., the Misinformation Specialist) have an equal probability of framing the initial argumentation. Over N turns, this enforced adversarial structure prevents the formation of conversational echo chambers and forces the models to genuinely challenge conflicting viewpoints.

3.5 Main Entry Point & Pipeline (`ollama_agent_test.py`)

Orchestrates the full credibility-assessment pipeline:

- (1) Selects and loads an article metadata JSON file (from a selected pool in a subdirectory) by user request at the terminal.
- (2) Extracts structured fields (RAG outputs, scores).
- (3) Assembles a structured prompt describing debate rules, field descriptions, and article data.
- (4) Executes the multi-agent debate.
- (5) Runs the post-debate summarization, judgment, and extraction workflow, concluding in a final reliability score out of 100.

3.6 Evaluation and Deterministic Verdict Extraction

Following the conclusion of the N -turn debate, the chaotic and lengthy transcript poses a challenge for direct scoring. LLMs tasked with simultaneous reasoning and strict numerical output formatting

frequently fail at one of the two constraints. To resolve this, the architecture employs a tiered, three-agent post-debate pipeline:

- (1) **Context Compression (Summarizer):** A neutral observer agent processes the full debate transcript and generates a synthesized summary message. This prevents the downstream Judge from becoming distracted by conversational noise or overly dominating agents.
- (2) **Decision Rendering (Judge):** The *Judge* agent receives the appended state array (‘transcript + summary’). Relieved of the burden of exact formatting, it is instructed purely to synthesize the arguments and produce a qualitative explanation ending in a quantitative assessment.
- (3) **Deterministic Parsing (Extractor):** Finally, an *Extractor* agent operates solely on the Judge’s verdict. Using a highly constrained system prompt, its only objective is to parse the Judge’s rationale and predictably isolate the numerical value into a standardized `##/100` format.

By conceptually separating reasoning (Judge) from formatting (Extractor), the system achieves a robust, deterministic programmatic output without sacrificing the nuanced “Chain-of-Thought” reasoning present in the debate.

4 POTENTIAL FUTURE WORK

While the current architecture automates a form of credibility assessments via multi-agent debate, there are several promising avenues for future research and system enhancement:

- **Real-Time Web Retrieval Tools:** Currently, the system relies on static metadata and pre-computed RAG features provided in the JSON input. Integrating live retrieval capabilities (e.g., web search APIs, direct database querying) would allow agents to actively fact-check emerging claims dynamically during the debate.
- **Hierarchical and Directed Debate Structures:** Exploring alternative orchestration strategies beyond flat shuffling or round-robin execution. Implementing structured hierarchical debate formats—where sub-agents debate specific claims and report back to a primary analytical agent—could improve the depth of the analysis.
- **Human-in-the-Loop Evaluation Baseline:** Conducting empirical evaluation of the system’s accuracy and ideological balance by comparing its extracted credibility scores (0–100) against expert annotations from professional human fact-checking organizations such as PolitiFact or Reuters.
- **Expanded Persona Topologies:** Broadening the scope of the instantiated agent pool (`agent_presets.py`) to include highly specialized domains (e.g., medical experts for health-related news, or legal analysts for policy news) can provide more nuanced scrutiny, specifically targeting domain-level misinformation.